Customer Care Process Automation

Two bots replacing two manual back-office jobs, with a rule about when not to decide.

One bot validates records against rules with known right answers, the other reads complaints and routes them to one of five back-end teams. TF-IDF and Naive Bayes reach 95.2% on 42 held-out tickets, and below 0.55 confidence the ticket goes to a human instead of being forced through. Forty-two tickets is a thin evaluation, and the 0.55 threshold was set by judgement about the cost of a needless escalation rather than swept.

What this evidences: critiquing model quality and evaluation design. A headline accuracy is not the result. What the agent does with the tickets it is unsure about is.

Taiwo Alabi · MSc Artificial Intelligence, National College of Ireland

---

Contents

1. The problem
2. Results
3. The process, before
4. The process, after
5. How the triage agent works
6. The decision that matters: when not to decide
7. The second bot: rule-based validation
8. Honest limitations
9. Running it
10. Terms used in this report

---

1. The problem

Hundreds of customer complaints sorted by hand every morning, on a Nigerian mobile network. A person reads each one and decides which back-end team it belongs to: SIM activation, network, refunds, payment and billing, or general.

Two things go wrong. People misroute tickets, because after the sixtieth complaint they all start to read the same. And the customer waits while their ticket finds the right desk, then waits again once it gets there.

A misroute is not a small error. It costs the customer days, and the business a second handling pass on work it already paid for once.

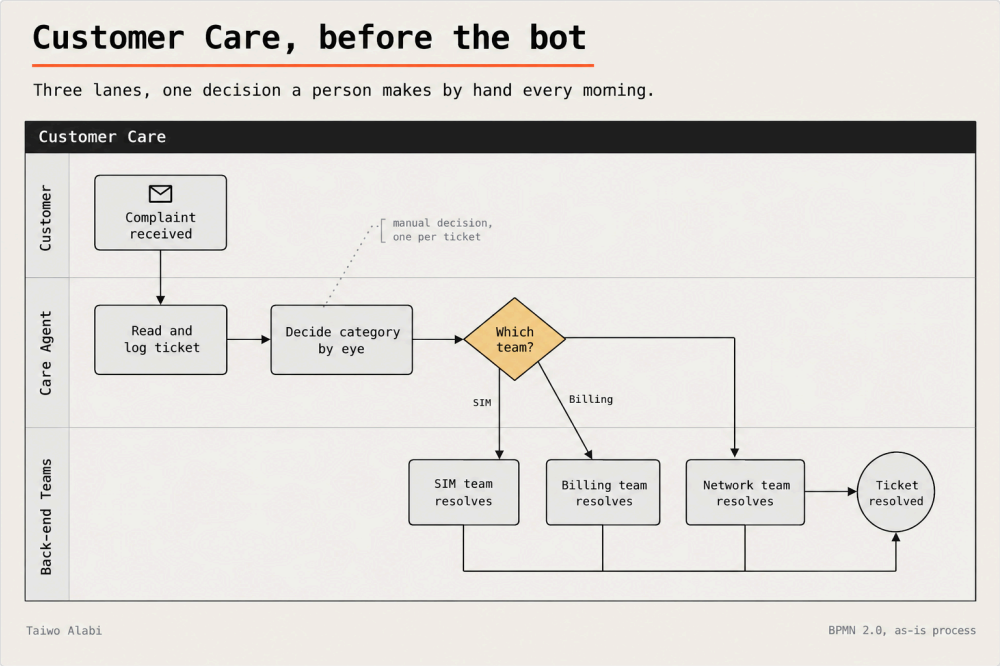
---

2. Results

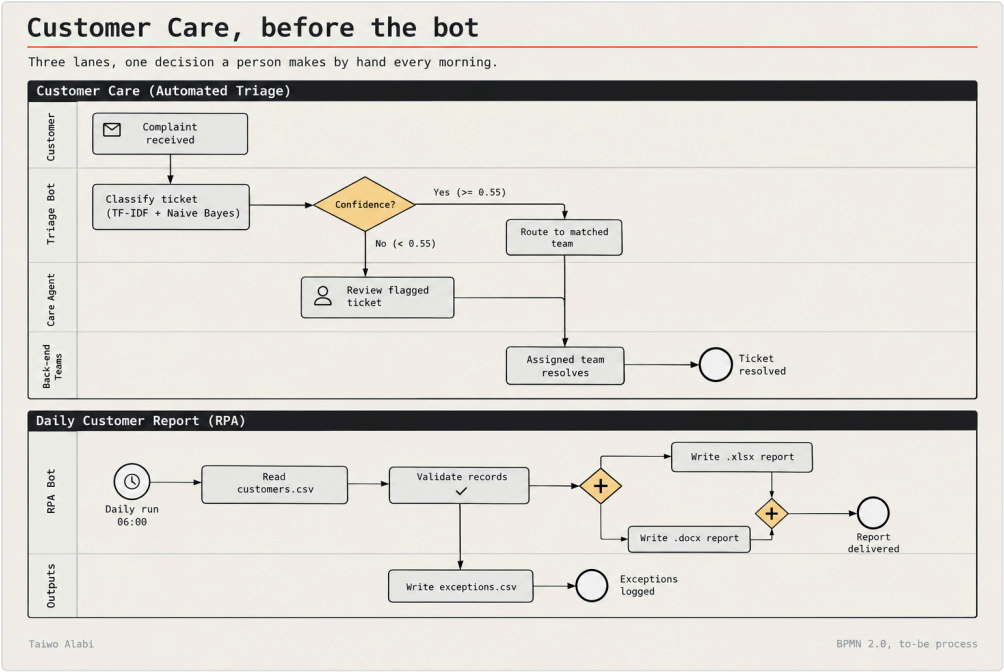
Measure	Result
Confidence threshold for escalation	0.55, below which a human decides
Routing accuracy on held-out tickets	95.2% (40 of 42)
Labelled corpus	210 tickets, 42 per category
Categories	5
Manual processes replaced	2

The accuracy number is the least interesting thing here. What matters is what the agent does with the tickets it is not confident about, which is section 6.

3. The process, before



Before the bot. Three lanes, one decision: a person reads the ticket and decides which of five back-end teams it belongs to. The bottleneck is the second swimlane, the manual decision.

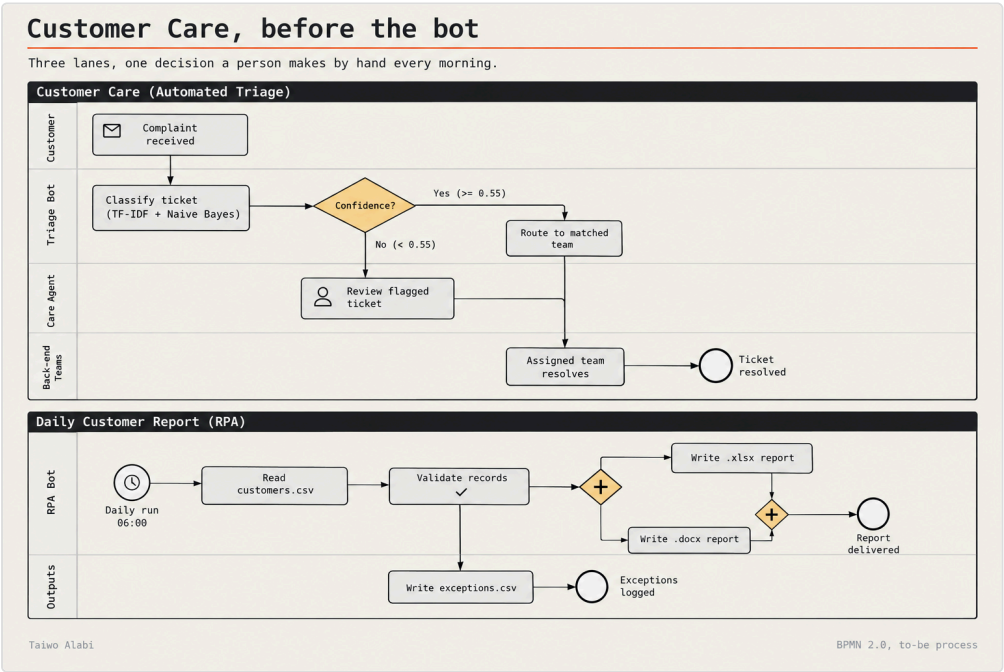


After the bot. The classifier proposes a team. The escalation gate, the diamond at 0.55 confidence, is what the project is really about. Tickets scoring below the line go to a human instead of being forced through.

I mapped out the existing process as a flowchart, using BPMN 2.0 (a standard way of drawing business processes), before writing any code. That order was deliberate: it meant the automation replaced a process I actually understood, rather than my assumption of one, and the model is what made the manual touchpoints explicit enough to decide which were worth automating.

---

4. The process, after



Two pools, two bots. The top pool is the classifier with the escalation gate. The bottom pool is the rule-based validation bot that runs the daily customer report.

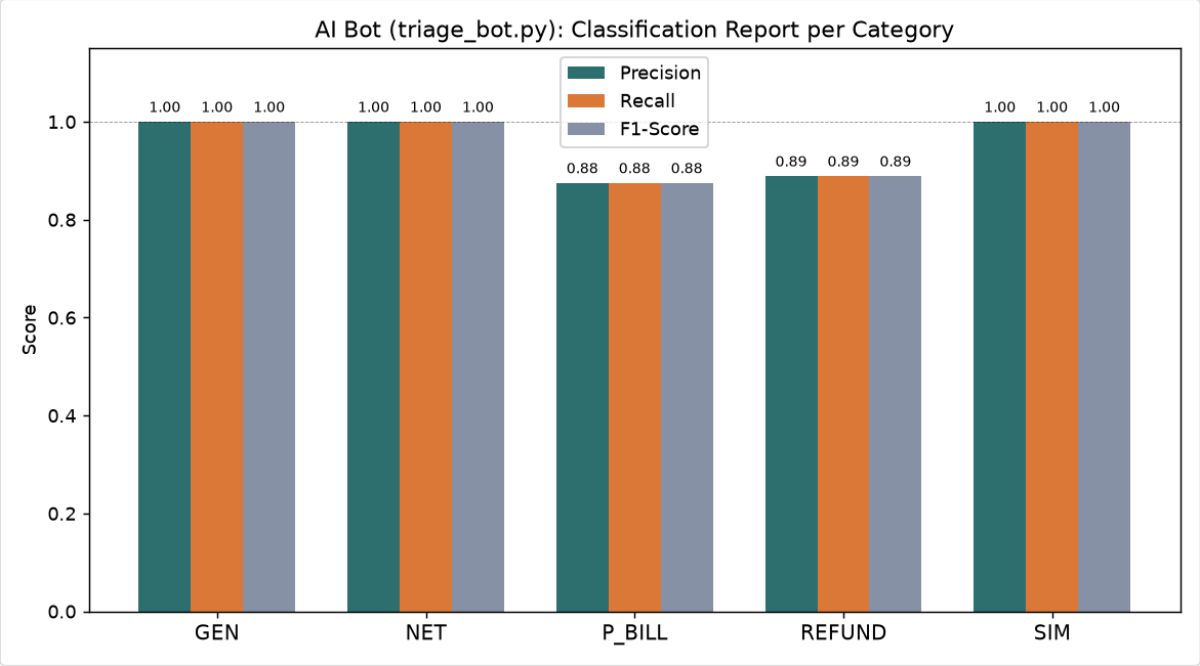
The redesign splits the work by the kind of decision each part requires. Field validation has known correct answers, so it goes to rules. Reading a complaint and inferring intent does not, so it goes to a classifier. The confidence gate sits between the classifier and the outcome.

---

5. How the triage agent works

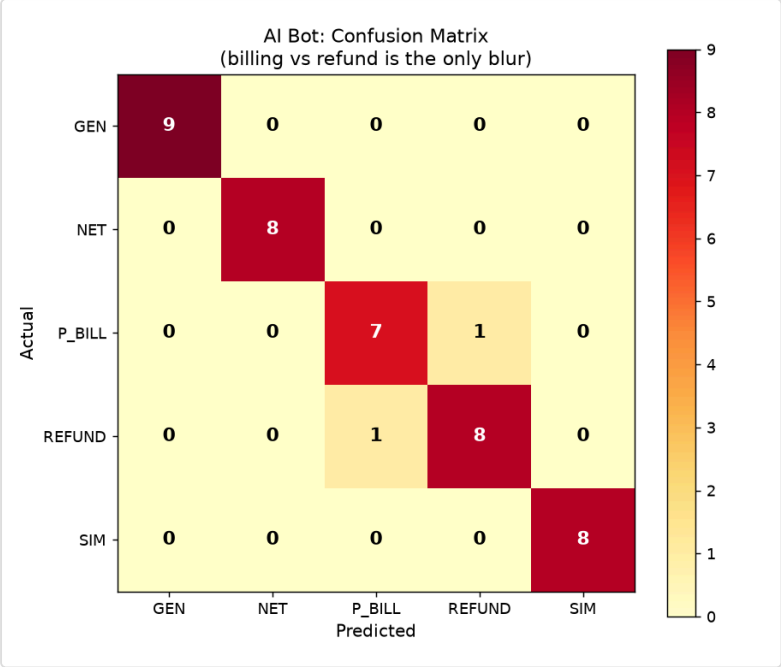
Ticket text is turned into numbers with TF-IDF, a way of scoring each word by how much it tells you about the topic, then sorted into one of the five routing categories by Naive Bayes, a simple, fast classifier built on word probabilities.

Naive Bayes is a deliberate choice rather than a default. The corpus is 210 tickets, which is small, and Naive Bayes stays reliable at that size where a heavier, more complex model would start memorising the training tickets instead of learning the pattern. It is also inspectable: I can see which words drive a routing decision, which matters when somebody asks why a ticket went where it did.



*Precision, recall and F1 per category on the 42 held-out tickets. P_BILL is the weak class at 0.88. The other four categories are at 1.00. The weakness is visible in the confusion matrix that follows.**Per-class precision, recall and F1: precision is how often a category the model picked was actually right, recall is how many of the true cases for that category it actually caught, and F1 blends the two into one number.*

Reported per category rather than as a single figure, because a router that is strong on common categories and weak on rare ones is a different system from one that is evenly good, and an average hides that difference.

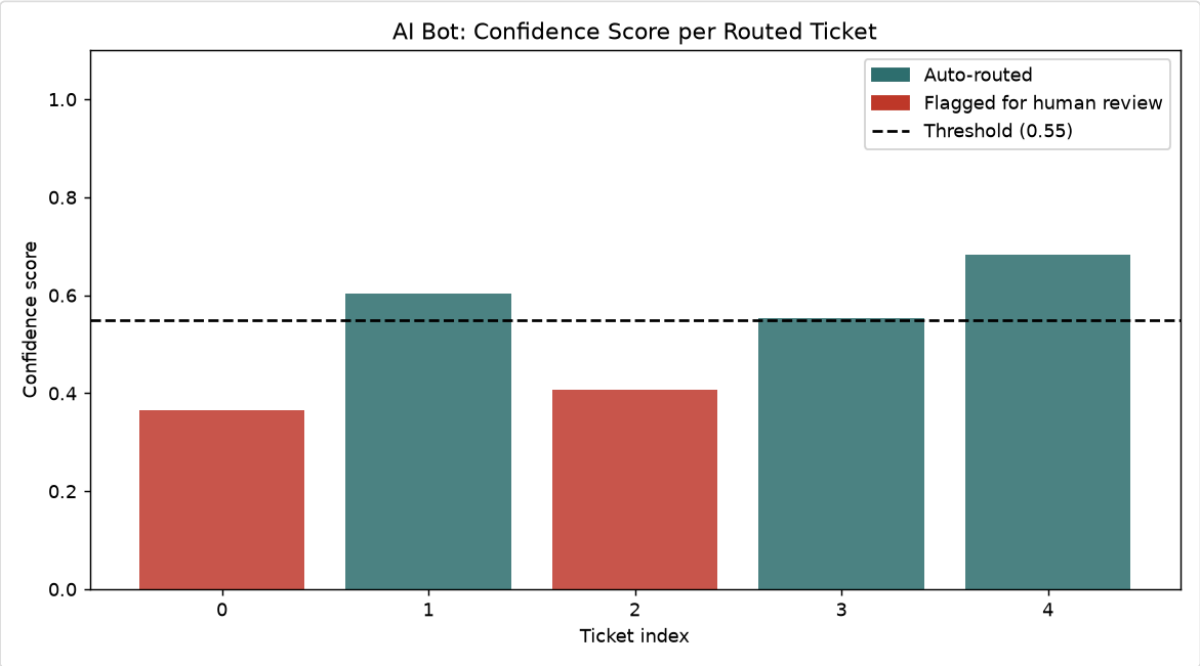


*Where the model loses. Two tickets, both P_BILL and REFUND, swap with each other. Every other prediction is on the diagonal. This is the failure mode the escalation gate is meant to absorb.**Where the classifier confuses one team with another.*

The matrix shows where errors concentrate. Confusion between neighbouring categories is a routing problem worth fixing; errors scattered evenly would suggest the features themselves are too weak, and that is not the pattern.

---

6. The decision that matters: when not to decide



Five tickets. Two of them (red, at 0.37 and 0.41) fall under the 0.55 line and go to a human. Three of them (teal, above the line) get auto-routed. The bar chart is the escalation decision in its rawest form.

A classifier will always return an answer. Asked to route a ticket it has never seen anything like, it will still pick the closest of the five categories and report a number. That is the failure mode worth designing against.

So the agent does not have to answer. Any ticket scoring below 0.55 confidence is flagged `needs_human_review` and escalated instead of routed.

The economics justify it. A needless escalation costs an agent about five minutes. A confident misroute costs the customer days. When one error is two orders of magnitude more expensive than the other, the correct system is one that escalates freely, and **I would rather report a lower automation rate than a higher misroute rate**.

---

7. The second bot: rule-based validation



The second bot, the rule-based one, ran on 120 records and flagged 5. Each of the five exceptions is named in the right panel: bad date, unknown segment, bad MSISDN prefix, bad MSISDN digits, missing field. The 4% exception rate is what the bot costs the team in manual review. **The validator checking each record against its requirements.*

The second automation produces the daily customer creation report: it reads the customer records, validates each field, writes the clean report to Excel and Word, and writes everything that failed validation to a separate exceptions file.

This half is deliberately not machine learning. These are hard business rules with known correct answers, so a classifier would introduce uncertainty where none needs to exist. Choosing not to use ML is as much a design decision as choosing to.

Outputs: `routed_tickets.csv`, `validation_exceptions.csv`, and the daily report in both `.xlsx` and `.docx`.

---

8. Honest limitations

- The corpus is small and synthetic. 210 generated tickets across five categories. Real complaints are messier: code-switching, abbreviations, several issues in one message. The generator was written to be realistic, but it is still a generator.
- 42 test tickets is a thin evaluation. 95.2% means 40 of 42, so a single ticket moves the figure by more than two points. The pattern of errors is more informative than the headline number.
- Five categories is a simplification. A real network has more teams and less clean boundaries between them.
- The confidence threshold was set by judgement, not swept. 0.55 reflects the cost asymmetry described above, but a proper sweep against real escalation costs would be the honest way to fix it.

---

9. Running it

```
python gen_data.py          # build the labelled corpus
python triage_bot.py        # train, evaluate, route
python rpa_report_bot.py    # validate records, write the daily report
python plot_results.py      # regenerate every figure in this document
```

Seeded, so results reproduce on every run.

Stack: Python · scikit-learn (TF-IDF, Naive Bayes) · pandas · BPMN 2.0 · openpyxl · python-docx

---

10. Terms used in this report

The terms below are defined the way this report uses them, for a reader who wants to check the reasoning rather than take it on trust.

- **Precision** - of the tickets the model sent to a category, how many actually belonged there. Low precision means it's wrong a lot when it says "this one".
- **Recall** - of the tickets that truly belonged to a category, how many the model actually caught. Low recall means it misses real cases.
- **F1 score** - one number that blends precision and recall, so a model can't look good by being great at only one of them.
- **TF-IDF** - a way of scoring each word in a ticket by how much it tells you about the topic, turning text into numbers a model can use.
- **Naive Bayes** - a simple, fast classifier that guesses a category based on which words showed up in similar tickets before.
- **BPMN 2.0** - a standard way of drawing a business process as a flowchart, before automating any of it.